

# Validador de Pedidos

## GoCase

---

Documentação Completa da Automação  
Agente de validação e organização de pedidos de fábrica

Business Case — Processo Seletivo de Estágio em RPA  
GoCase (GoGroup) — Extrema/MG

Data: 18/07/2026  
Área de negócio: Operações de Fábrica

## 1. Sumário Executivo

Este documento descreve o **Validador de Pedidos GoCase**, uma automação em Python que atua como filtro de qualidade entre a captação de pedidos e o chão de fábrica. O sistema lê a planilha de pedidos exportada do e-commerce, rejeita os pedidos inconsistentes explicando o motivo de cada rejeição, prioriza os pedidos válidos pelo aperto do prazo de entrega e gera relatórios Excel prontos para a equipe de produção agir.

A dor atacada é concreta: em uma operação de produção sob demanda, pedidos com dados quebrados (cliente sem nome, e-mail inválido, quantidade zerada, valor que não fecha, prazo vencido, pedido duplicado) chegam à fábrica e geram retrabalho, desperdício de material personalizado e atraso de entrega. A conferência manual desses dados é lenta, cansativa e sujeita a falha humana.

A solução processa um lote de 50 pedidos em menos de um segundo, com resultado determinístico e auditável, e foi construída em dois modos de operação: **standalone** (terminal ou duplo-clique, para o operador) e **MCP Server** (chamável por qualquer ferramenta de IA, para integração futura com o ecossistema da empresa).

### Resultado de uma execução de demonstração

| Métrica             | Valor                           |
|---------------------|---------------------------------|
| Pedidos processados | 50                              |
| Pedidos válidos     | 40 (80%)                        |
| Pedidos rejeitados  | 10 (20%)                        |
| Tempo de execução   | < 0,05 s                        |
| Relatórios gerados  | 3 planilhas Excel + log técnico |

## 2. Contexto de Negócio e a Dor

A GoCase fabrica produtos personalizados sob demanda — cases, mochilas, garrafas, capas. Cada pedido vira uma ordem de produção física e individualizada. Diferente de um estoque padronizado, um pedido com dado errado não é apenas um registro incorreto: é material gasto, hora-máquina perdida e um cliente que não recebe.

### De onde vêm os erros

Pedidos entram por múltiplos canais (site, marketplace, B2B, loja física), cada um com seu nível de validação na origem. O resultado é uma planilha heterogênea onde convivem pedidos perfeitos e pedidos com falhas de captação. Os erros mais comuns e custosos são:

- **Cadastro incompleto** — cliente sem nome, e-mail malformatado (impede notificação de rastreo).
- **Quantidade inválida** — zero ou negativa, vinda de erro de importação.
- **Valor incoerente** — valor\_total que não bate com quantidade × valor unitário, sinal de erro de cálculo ou fraude.

- **Prazo vencido** — pedido que já entra atrasado e precisa de tratamento imediato, não de produção normal.
- **Duplicidade** — o mesmo pedido importado duas vezes, que produziria o item em dobro.

### O custo de não validar

Sem um filtro automático, a conferência é feita a olho, pedido a pedido. Estimando um tempo conservador de conferência manual dos nove pontos de cada pedido, um lote de 50 pedidos consome tempo significativo de um operador — tempo que se repete a cada lote, todo dia. Além do custo de tempo, a conferência manual deixa passar erros sutis, como uma diferença de centavos no valor total ou uma duplicata separada por dezenas de linhas.

## 3. Objetivo e Escopo

### Objetivo

Automatizar a validação e a priorização de pedidos antes da produção, entregando à fábrica apenas pedidos consistentes, já ordenados por urgência, e devolvendo à gestão a lista de pedidos barrados com o motivo exato de cada um.

### Dentro do escopo

- Leitura e tipagem da planilha de pedidos.
- Validação por nove regras de negócio, com acúmulo de múltiplos motivos por pedido.
- Classificação e ordenação dos válidos por prioridade de prazo.
- Geração de três relatórios Excel formatados e de um resumo consolidado.
- Operação em dois modos: standalone e MCP Server.

### Fora do escopo (evolução futura)

- Escrita de volta no ERP ou no e-commerce (hoje a saída é relatório).
- Notificação ativa por Slack/e-mail (a arquitetura já prevê, ver seção 16).
- Interface web — o operador usa terminal/planilha ou uma IA via MCP.

## 4. Visão Geral da Solução

A automação é um **pipeline de cinco etapas** orquestrado por um agente. Cada etapa tem responsabilidade única e é isolada das demais: se uma falha, o erro é registrado e as etapas seguintes que ainda fizerem sentido continuam, evitando que um problema pontual derrube o lote inteiro.

| Etapa          | O que faz                                        | Módulo         |
|----------------|--------------------------------------------------|----------------|
| 1. Leitura     | Lê o Excel e tipa as colunas (datas, números)    | leitor.py      |
| 2. Validação   | Aplica as 9 regras; separa válidos de rejeitados | validador.py   |
| 3. Organização | Calcula prioridade e ordena os válidos           | organizador.py |
| 4. Relatórios  | Gera os 3 Excel formatados                       | relatorio.py   |

| Etapa          | O que faz                            | Módulo         |
|----------------|--------------------------------------|----------------|
| 5. Notificação | Monta e publica o resumo da execução | integracoes.py |

O fluxo de dados é direto: a planilha vira um DataFrame; a validação o divide em `df_validos` e `df_rejeitados` (este último ganha a coluna `motivo_rejeicao`); a organização enriquece os válidos com `dias_restantes` e `prioridade`; e os relatórios materializam tudo em disco.

## 5. Arquitetura

O projeto segue responsabilidade única por módulo — cada arquivo faz uma coisa e é testável isoladamente. Essa separação foi escolhida para que a evolução futura (trocar a fonte de dados, adicionar notificações) toque apenas um ponto do código.

| Módulo           | Responsabilidade                                                                     |
|------------------|--------------------------------------------------------------------------------------|
| config.py        | Carrega config.yaml (regras externas) com fallback embutido; nunca quebra.           |
| gerar_dados.py   | Fixture de teste/demo: gera planilha simulada com erros propositais. NÃO é produção. |
| leitor.py        | Lê o Excel, tipa colunas e valida a presença das colunas esperadas.                  |
| validador.py     | Aplica as 9 regras; separa válidos de rejeitados; acumula motivos.                   |
| organizador.py   | Calcula dias_restantes e prioridade; ordena os válidos.                              |
| relatorio.py     | Gera os 3 Excel formatados (cores, bordas, freeze, larguras).                        |
| integracoes.py   | Abstrai entrada (LeitorDados) e saída (Notificador) para conectores MCP.             |
| assistente_ia.py | Camada de IA: prepara rejeitados e aplica correções propostas (ver seção 11).        |
| agente.py        | Orquestra o fluxo, configura logging e monta o resumo.                               |
| main.py          | Entrada standalone (terminal / .bat).                                                |
| mcp_server.py    | Entrada MCP Server (stdio, 5 tools + 1 prompt).                                      |

### A camada de integrações — pensada para o futuro

Hoje a entrada é um arquivo Excel e a notificação é um log. Mas a interface (`ler()`, `enviar_resumo()`, `enviar_alerta()`) foi desenhada para que **apenas a implementação mude** quando a fonte virar Google Sheets, ERP ou banco de dados, e a notificação virar Slack ou e-mail — conectados via MCP. O restante do agente permanece intacto. É o ponto de extensão que transforma este business case em algo plugável ao ecossistema real da empresa.

## 6. Regras de Validação

Cada pedido é avaliado contra **todas** as regras — um pedido pode acumular vários motivos, concatenados por “; ” na coluna `motivo_rejeicao`. Isso dá à gestão a lista completa de

problemas de cada pedido de uma só vez, em vez de um erro por vez a cada reprocessamento.

| # | Campo          | Regra                                                                 |
|---|----------------|-----------------------------------------------------------------------|
| 1 | id_pedido      | Não vazio e não duplicado. Na duplicata, a 2ª ocorrência é rejeitada. |
| 2 | cliente        | Não vazio nem só espaços.                                             |
| 3 | email          | Formato válido: texto@texto.dominio.                                  |
| 4 | quantidade     | Número inteiro positivo (> 0).                                        |
| 5 | valor_unitario | Número positivo (> 0).                                                |
| 6 | valor_total    | Bate com quantidade × valor_unitario (tolerância R\$ 0,02).           |
| 7 | prazo_entrega  | Não pode estar no passado.                                            |
| 8 | produto        | Não vazio.                                                            |
| 9 | sku            | Não vazio.                                                            |

Exemplo de motivo com múltiplos erros: Cliente vazio; Email inválido: formato incorreto.

## 7. Classificação de Prioridade

Os pedidos válidos recebem `dias_restantes = (prazo_entrega - hoje)` e são classificados em quatro faixas. A ordenação final coloca os URGENTE primeiro e, dentro de cada faixa, o prazo mais apertado antes — a fila que a fábrica deve seguir.

| Prioridade | Dias até o prazo | Cor no relatório |
|------------|------------------|------------------|
| URGENTE    | 0 a 2 dias       | Vermelho claro   |
| ALTA       | 3 a 5 dias       | Laranja claro    |
| NORMAL     | 6 a 10 dias      | Verde claro      |
| BAIXA      | 11 dias ou mais  | Sem cor          |

## 8. Formato de Dados

### Entrada — `data/pedidos_entrada.xlsx`

Uma linha por pedido, com as colunas abaixo. A falta de qualquer coluna faz o leitor parar cedo com mensagem clara dizendo qual coluna faltou.

| Coluna      | Tipo  | Exemplo      |
|-------------|-------|--------------|
| id_pedido   | texto | PED-00001    |
| data_pedido | data  | 14/07/2026   |
| cliente     | texto | Ana Carolina |

| Coluna         | Tipo    | Exemplo                            |
|----------------|---------|------------------------------------|
| email          | texto   | ana.carolina@gmail.com             |
| produto        | texto   | Case iPhone 15                     |
| sku            | texto   | CSE-IPH15-4821                     |
| quantidade     | inteiro | 3                                  |
| valor_unitario | número  | 89,90                              |
| valor_total    | número  | 269,70                             |
| canal          | texto   | site                               |
| endereco       | texto   | Rua das Flores, 123 — São Paulo/SP |
| prazo_entrega  | data    | 25/07/2026                         |

## Saída — pasta output/

**Para o operador** — as três planilhas que interessam ao chão de fábrica:

- **pedidos\_validados.xlsx** — aprovados, linhas coloridas por prioridade, valores formatados, cabeçalho congelado.
- **pedidos\_rejeitados.xlsx** — com erro, coluna motivo\_rejeicao em vermelho/negrito.
- **resumo\_execucao.xlsx** — métricas consolidadas (totais, %, por prioridade, por canal, valores, tempo).

**Artefatos internos** — isolados em output/\_sistema/, fora da visão do operador: o log técnico da execução e um cache JSON do último resumo (usado pela tool MCP consultar\_resumo). Assim o operador abre output/ e vê apenas as três planilhas que importam.

## 9. Modos de Operação

### 9.1. Terminal (avaliação e uso técnico)

```
pip install -r requirements.txt --break-system-packages
python main.py
```

*Se a planilha de entrada não existir, o sistema gera automaticamente uma planilha de exemplo e a processa — comportamento intencional para esta entrega, que permite rodar o projeto com um único comando. Em produção real esse gatilho seria removido em favor de uma falha explícita (ver seção 16).*

### 9.2. Duplo-clique (operador não-técnico)

O arquivo **Validar Pedidos.bat** executa a automação sem nenhum comando: o operador coloca a planilha em data/, dá dois cliques, e a janela processa, mostra o resumo e fica aberta. Os relatórios aparecem em output/.

### 9.3. MCP Server (integração com IA)

```
python mcp_server.py
```

O servidor fala o protocolo MCP (stdio, JSON-RPC) e expõe cinco ferramentas e um prompt guia, chamáveis por qualquer IA compatível (Claude Desktop, Cursor, n8n):

| Tool                    | Função                                                   |
|-------------------------|----------------------------------------------------------|
| gerar_dados_exemplo     | Gera a planilha simulada (uso de teste/demo).            |
| validar_pedidos         | Executa o fluxo completo e retorna o resumo formatado.   |
| consultar_resumo        | Retorna o resumo da última execução, sem reprocessar.    |
| analisar_rejeitados     | Prepara os rejeitados para a IA corrigir (ver seção 11). |
| revalidar_com_correcoes | Aplica as correções da IA e revalida o lote.             |

Conectado ao Claude Desktop, o operador pede em linguagem natural — “valide os pedidos da planilha” — e a IA aciona a tool. Ele nunca vê o protocolo. As duas últimas tools formam a camada de correção assistida detalhada na seção 11.

## 10. Configuração Externa (config.yaml)

As regras de negócio ficam **fora do código**, num arquivo `config.yaml`. Um gestor não-técnico ajusta limites sem abrir Python nem pedir um redeploy — muda o YAML e roda de novo.

### O que é configurável

- **tolerancia\_valor** — diferença aceita entre `valor_total` e `quantidade × valor_unitario` (padrão R\$ 0,02).
- **regex\_email** — padrão de validação do e-mail.
- **colunas\_obrigatorias** — quais colunas a planilha deve conter.
- **faixas de prioridade** — os intervalos de dias de URGENTE, ALTA, NORMAL e BAIXA, e a própria ordem da fila.

**Segurança por padrão:** se o arquivo faltar ou uma chave estiver ausente, o sistema cai nos valores-padrão embutidos em `src/config.py` e registra um aviso — nunca quebra por configuração incompleta. Assim, ajustar uma regra é seguro e reversível: basta editar o YAML.

## 11. Camada de IA — Correção Assistida via MCP

Além de barrar pedidos, o sistema ajuda a **recuperá-los**. A camada de IA fecha o ciclo **rejeição** → **correção** → **aprovação** usando a IA já conectada via MCP como o cérebro do processo.

### Princípio: sem modelo embutido, sem chave de API

O servidor não embute um LLM próprio nem exige credenciais. Ele prepara os pedidos rejeitados num formato que a IA conectada (Claude Desktop, por exemplo) consegue interpretar, e aceita de volta as correções propostas. A inteligência vive no cliente MCP; o servidor fornece contexto e revalida.

### Divisão de trabalho

- **Erros mecânicos** (valor\_total que não fecha, espaços sobrando, e-mail a normalizar) recebem uma sugestão determinística, resolvida por regra — sem IA.
- **Erros semânticos** (nome digitado errado, e-mail com domínio faltando, quantidade implausível) ficam para a IA, que infere a intenção.
- **Dados ausentes por completo** (cliente totalmente vazio) são sinalizados para revisão humana, nunca inventados.

### O fluxo (tools analisar\_rejeitados e revalidar\_com\_correcoes)

1. A IA chama analisar\_rejeitados e recebe cada pedido barrado, seus motivos e as sugestões mecânicas já resolvidas.
2. A IA propõe as correções semânticas.
3. A IA envia tudo para revalidar\_com\_correcoes, que aplica as correções à planilha, revalida o lote inteiro e relata quantos pedidos foram recuperados. O prompt corrigir\_pedidos\_rejeitados orquestra esse roteiro.

*Exemplo real medido: dois e-mails inválidos corrigidos pela IA reduziram os rejeitados de 10 para 8 e elevaram os válidos de 40 para 42, num único ciclo.*

**Degradação graciosa:** se a IA ou o MCP não estiverem disponíveis, nada quebra — os rejeitados continuam saindo no Excel com o motivo, e a gestão corrige manualmente como antes. A camada de IA é um ganho aditivo, não uma dependência do núcleo.

## 12. Instalação e Uso — Passo a Passo

---

1. Instalar Python 3.10 ou superior.
2. Instalar as dependências:  
`pip install -r requirements.txt --break-system-packages`
3. (Opcional) Colocar a planilha real em data/pedidos\_entrada.xlsx.
4. (Opcional) Ajustar regras em config.yaml.
5. Executar: `python main.py` — ou dar dois cliques em Validar Pedidos.bat.
6. Abrir os relatórios na pasta output/.

*As pastas data/ e output/ são criadas automaticamente pelo código — não precisam existir de antemão nem ser versionadas.*

## 13. Decisões de Design

---

Escolhas de engenharia que não são óbvias apenas lendo o código, registradas aqui para o avaliador entender o raciocínio por trás delas:

- **Log em stderr, não stdout.** No modo MCP o stdout carrega o protocolo JSON-RPC; qualquer log ali corromperia a comunicação e quebraria o cliente. O stderr fica livre para diagnóstico e o console segue legível.
- **Reconfiguração para UTF-8.** O console Windows usa cp1252 e corrompe acentos; os streams são reconfigurados para UTF-8, e os arquivos de log já são gravados em UTF-8.
- **Prazo contado a partir de hoje.** Se o prazo fosse data\_pedido + N dias, pedidos antigos cairiam no passado por acidente. Contar de hoje mantém a coerência e distribui as faixas de prioridade.



- **Tolerância de R\$ 0,02 no valor\_total.** Soma de floats acumula erro de arredondamento; sem a tolerância, pedidos corretos gerariam falso positivo.
- **Tipagem com errors='coerce'.** Valores inválidos viram NaN/NaT na leitura em vez de estourar — a decisão de rejeitar fica com o validador, não com o leitor.
- **Fixture de dados separada da produção.** O gerador de exemplo é ferramenta de teste e não tem lançador próprio, para o operador nunca sobrescrever dados reais com dados falsos por engano.
- **Camada de integrações abstrata.** Entrada e saída atrás de interfaces, prontas para virar conectores MCP sem tocar o núcleo do agente.
- **Regras externas em config.yaml.** Limites e faixas fora do código, para o gestor ajustar sem redeploy, com fallback embutido que impede quebra por config faltante.
- **IA como cliente, não como dependência.** A camada de correção usa a IA conectada via MCP; o núcleo funciona igual sem ela (degradação graciosa).

## 14. Testes e Qualidade

O arquivo `testar.py` roda o fluxo completo e verifica oito pontos, servindo como teste de fumaça antes de qualquer entrega:

- Planilha de entrada gerada (50 pedidos).
- Agente executa sem exceção.
- `pedidos_validados.xlsx` existe e tem linhas.
- `pedidos_rejeitados.xlsx` existe e tem linhas.
- `resumo_execucao.xlsx` existe.
- `log_execucao.log` existe e não está vazio.
- Consistência: válidos + rejeitados = total processado.
- Todos os rejeitados têm motivo de rejeição preenchido.

Padrões de código aplicados em todo o projeto: Python 3.10+, encoding UTF-8 em toda operação de arquivo, type hints em todas as funções, docstrings em português, logging consistente (sem `print()` solto fora do teste) e `pathlib.Path` em vez de strings de caminho.

## 15. Ganhos Esperados

| Dimensão                                   | Antes (manual)                    | Depois (automação)                |
|--------------------------------------------|-----------------------------------|-----------------------------------|
| Tempo por lote de 50 pedidos               | Dezenas de minutos de conferência | Menos de 1 segundo                |
| Consistência                               | Varia com o operador e o cansaço  | Determinística e repetível        |
| Erros sutis (centavos, duplicata distante) | Passam despercebidos              | Detectados por regra              |
| Rastreabilidade                            | Sem registro do motivo            | Motivo explícito por pedido + log |
| Priorização de produção                    | Feita a olho                      | Automática, por aperto de prazo   |

| Dimensão                  | Antes (manual)           | Depois (automação)                |
|---------------------------|--------------------------|-----------------------------------|
| Recuperação de rejeitados | Correção manual, um a um | Correção assistida por IA via MCP |

Além do ganho de tempo direto, a automação **libera o operador** da conferência repetitiva para tarefas de maior valor, **reduz desperdício** de material ao barrar pedidos quebrados antes da produção, e **dá à gestão visibilidade** imediata sobre a qualidade dos pedidos por canal.

## 16. Limitações e Evolução Futura

---

### Já implementado nesta versão

- **Configuração externa** (config.yaml) — regras editáveis sem tocar no código.
- **Camada de IA** — correção assistida de rejeitados via MCP, com degradação graciosa.

### Limitações atuais

- A entrada é um arquivo Excel; ainda não lê direto do ERP/e-commerce.
- A saída é relatório; não escreve de volta nos sistemas de origem.
- A geração automática de dados em main.py é conveniência de demo, não de produção.

### Roadmap de evolução

- **Conectores MCP de entrada** — ler pedidos direto de Google Sheets, ERP ou banco, trocando apenas a implementação de LeitorDados.
- **Notificação ativa** — enviar o resumo e alertas por Slack/e-mail via Notificador.
- **Escrita de retorno** — marcar no sistema de origem o status de cada pedido.
- **Estado entre execuções** — deduplicar entre lotes e guardar histórico de qualidade.
- **Modo produção** — substituir a geração automática por falha explícita quando a planilha estiver ausente.

---

*Documento gerado para o business case do processo seletivo de Estágio em RPA — GoCase (GoGroup). O plano de ação para implementação em produção (cronograma, custos e antecipação de desafios) é entregue em documento à parte.*